46-926, Spring 2019: Homework #1

Due 5PM, Monday, January 28, 2019.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

(Because this is a two-week homework, I'm reserving the right to add another problem or so that I'm currently debating. I'll do this by Thursday, Jan 17 if I do.)

0. Get python, ipython, jupyter set up on your machine. Install `scipy`, `numpy`, `sklearn`, `statsmodels`. These will be useful for this homework and in the future, so let's get it out of the way. *No need to submit anything for this problem. It's just a good thing to do.*

1. *Error measures and prediction.* We're going to consider a very, very simple prediction problem.

   Suppose that you know I'm going to draw data points Y from an Exponential(1) distribution. You're going to try to predict the data points that I draw, using two possible estimators:

   I. The mean of an Exponential(1) (which happens to be 1)

   II. The median of an Exponential(1) (which happens to be log(2)).

   (a) Simulate 1000 points from an Exponential(1). What is the mean squared prediction error of estimators I and II? That is: $\frac{1}{n}\sum_{i=1}^{n}(Y_i - \widehat{y})^2$, where $\widehat{y} \in \{1, \log(2)\}$. Which estimator does better for this error measure?

   (b) Simulate 1000 points from an Exponential(1). What is the mean absolute prediction error of estimators I and II? That is: $\frac{1}{n}\sum_{i=1}^{n}|Y_i - \widehat{y}|$, where $\widehat{y} \in \{1, \log(2)\}$. Which estimator does better for this error measure?

> **Hint:** You can generate random exponentials with **scipy.stats.expon**.

2. *Asymmetric loss and the regression function.* In class, we saw that the best possible estimator of Y when minimizing expected squared error in regression (if you know the distribution of X, Y) is the conditional mean, $\mathbb{E}(Y|X)$. This is called the *regression function*. In this problem, we will see how this result can change when you have a loss function other than squared error.

   Define the loss function

$$\mathcal{L}(Y, f(X)) = b\left(e^{a(Y-f(X))} - a(Y - f(X)) - 1\right),$$

   with $a, b > 0$ and $f(X)$ representing the estimate of Y.

   (a) Plot the loss function against $z = Y - f(X)$ for $z \in [-2, 2]$, $a = 1.1$, $b = 2$. Describe what you see. Why might you use a loss function like this?

   (b) Find the optimal rule, $f(x)$, for minimizing the expected loss. Assume you can use the expectation of any function of Y conditional on X.

   Hints:

   - Follow the basic approach of the derivation from class: Get a nested conditional expectation of X, pass it through as many terms as possible, and take a derivative of the inside of the outer expectation with respect to $f(x)$.
   - Expect something that reminds you of a moment generating function.

   (c) Suppose that you know the conditional distribution of $Y|X = x$ is $N(\beta \cdot x, \sigma^2)$, for known scalars $\beta, \sigma \in (0, \infty)$. What is your optimal estimator $f(x)$?

   Useful fact: You know from your probability course that for a Gaussian random variable $Z \sim N(\mu, \sigma^2)$, the Moment Generating Function is:

$$\mathbb{E}(e^{tZ}) = e^{\mu t + \frac{1}{2}\sigma^2 t^2}$$

   for any $t \in \mathbb{R}$.

   (d) There is code on blackboard (`asymm_loss.py`) for this problem. Change the body of the `f_yours` function to correspond to your estimator from the previous part. The code will run simulations to compare the average loss of your estimator to the average loss of the conditional mean (βx).

   What is your average loss in the simulation? Do you have a lower average loss than the conditional mean? Why do you think this is, thinking back to the plot of the loss function and the form of your estimator.

3. *What happens to linear regression in high dimensions?* Here we will replicate the simulation from Lecture 1. Later in class/homeworks we will add to this to see that we can improve on least squares.

   Simulate the following setting: To generate each data point, generate independent variables $X_1, ..X_p \sim N(0,1)$, and let $Y = 4X_1 + \varepsilon$, where $\varepsilon \sim N(0,1)$. This means that the linear model assumption is *true* for this data for any number of variables p, though there are many useless additional variables when p is large.

   Holding n fixed at 100 observations, vary the number of variables p from 2 to 80. For each setting, run 100 simulations of:

   (i) Draw $n = 100$ data points

   (ii) Fit a linear regression of Y on $X_1, \ldots, X_p$ using these data points. Call the resulting coefficient vector $\widehat{\beta}$.

   (iii) Draw a separate test set of $m = 100$ data points, compute predictions at these points using your estimated $\widehat{\beta}$, and compute mean squared prediction error of these points:
   $\frac{1}{m}\sum_{i=1}^{m}(y_i - x_i^T\widehat{\beta})^2$

   Plot average (over simulations) mean squared prediction error vs. p. What do you see?

   Hint: You may find some of the following functions useful: `sklearn.linear_model.LinearRegression`, `statsmodels.OLS`, and `sklearn.metrics.mean_squared_error`.

4. Given a response vector $y \in \mathbb{R}^n$, predictor matrix $X \in \mathbb{R}^{n \times p}$, and tuning parameter $\lambda \geq 0$, recall the ridge regression estimate

$$\widehat{\beta}^{\text{ridge}} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} \ \|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2.$$

   (a) Show that $\widehat{\beta}^{\text{ridge}}$ is simply the vector of linear regression coefficients from regressing the response $\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+p}$ onto the predictor matrix $\tilde{X} = \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix} \in \mathbb{R}^{(n+p)\times p}$, where here $0 \in \mathbb{R}^p$, and $I \in \mathbb{R}^{p \times p}$ is the identity matrix.

   (b) Show that when $\lambda > 0$, the matrix $\tilde{X}$ always has full column-rank, i.e., its columns are always linearly independent, regardless of the columns of X. Hence argue that the ridge regression estimate is always unique, for any matrix of predictors X.

(c) Write out an explicit formula for $\widehat{\beta}^{\text{ridge}}$ involving X, y, λ. Conclude that for any $a \in \mathbb{R}^p$, the estimate $a^T \widehat{\beta}^{\text{ridge}}$ is a linear function of y. **Hint:** Take advantage of part (a), instead of doing any calculus or real work.

Sidenote: Among other things, it is interesting that the ridge regression solution is a linear function of Y, because this means that if it were unbiased, it would be covered by the Gauss-Markov theorem. This means that if it were not biased, it could not have lower variance than Least squares, so would lose in prediction error.

It is also worth noting that unlike Ridge regression, the Lasso is not a linear function of Y.